

Data Engineering

End-to-end foundations: ingestion, storage, modeling, pipelines, quality, security, testing, and architecture

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

1. What Data Engineering Is

Data engineering is the discipline of integrating, transforming, and consolidating data into reliable structures for analytics, applications, and machine learning.

Key idea: A data engineer builds systems that make data available, correct, timely, understandable, secure, and cost-effective.

Responsibilities

Typical responsibilities include ingestion, storage, transformation, orchestration, data modeling, quality, governance, monitoring, and platform reliability.

- Build repeatable pipelines.
- Design data stores and schemas.
- Automate operational work.
- Partner with analysts, scientists, application teams, and business stakeholders.

The data lifecycle

Data moves through a lifecycle from generation to ingestion, storage, processing, serving, archival, and eventual deletion.

- Every stage needs ownership and observability.
- Retention and deletion are part of engineering, not an afterthought.
- Lineage helps explain how a downstream metric was produced.

Engineering mindset

Data engineering combines software engineering with distributed systems, databases, cloud infrastructure, and domain understanding.

- Correctness comes before optimization.
- Automation beats manual recurring work.
- Design for failure and recovery.

2. Data Sources and Ingestion Patterns

Ingestion moves data from source systems into a platform where it can be processed. Sources may be relational databases, files, APIs, SaaS applications, message brokers, sensors, or application events.

Key idea: Choose an ingestion pattern based on change frequency, source capabilities, latency requirements, volume, and correctness needs.

Batch

Batch ingestion processes a bounded set of data on a schedule or on demand. It is simple and often sufficient for daily/hourly analytics.

- Full load is easy but can be expensive.
- Incremental load processes only new/changed records.
- Backfills should be explicitly supported.

Streaming

Streaming ingestion processes events continuously or in micro-batches. It is useful when low latency matters.

- Event time and ordering matter.
- Checkpointing and replay are core operational concepts.
- Streaming does not automatically mean real-time business value.

CDC

Change Data Capture captures inserts, updates, and deletes from source systems. It enables incremental replication while preserving changes.

- A reliable change sequence is essential.
- Deletes must be represented correctly.
- Schema changes need handling.

API and file ingestion

APIs often impose rate limits and pagination. File ingestion must handle naming conventions, late arrivals, duplicates, corrupt files, and schema drift.

- Store raw inputs when auditability matters.
- Track ingestion metadata.
- Make ingestion idempotent.

3. ETL vs ELT and Pipeline Architecture

ETL means extract-transform-load; ELT means extract-load-transform. Modern cloud platforms often favor ELT because scalable compute can transform data after landing it in inexpensive storage.

Key idea: ETL and ELT are patterns, not competing religions. Choose based on security, source constraints, transformation engines, cost, and data governance.

ETL

Data is transformed before being loaded into the target store. This can reduce the amount of raw data entering a constrained target or satisfy transformation-at-ingest requirements.

- Useful when the destination expects curated data only.
- Can complicate reprocessing if raw data is not retained.

ELT

Data is loaded first, then transformed in the target/lakehouse environment. This supports reprocessing and exploratory workloads.

- Keep raw data governed and protected.
- Transformation compute can scale independently.
- Retaining raw data improves recovery but increases storage/governance needs.

Pipeline layers

A common architecture separates ingestion, raw storage, standardized/curated data, and business-serving data.

- Raw layer prioritizes fidelity.
- Curated layer prioritizes consistency.
- Serving layer prioritizes consumer performance and usability.

4. Data Storage: Files, Object Storage, and Databases

Different storage technologies optimize for different access patterns. Data engineers should distinguish object storage, relational databases, analytical warehouses, lakehouses, and caches.

Key idea: Storage choice should follow workload: transaction processing, analytical scans, streaming, archival, or serving.

Object storage

Object stores are scalable repositories for files and large datasets. They are common foundations for data lakes.

- Use durable formats such as Parquet for analytics.
- Design paths and metadata carefully.
- Object storage is not automatically a relational database.

Relational databases

Relational databases provide transactions, constraints, indexes, joins, and SQL. They are strong for operational systems and many structured workloads.

- Normalize operational schemas when appropriate.
- Use indexes based on access patterns.
- Separate OLTP and analytical workloads when necessary.

Warehouse/lakehouse

Warehouses optimize analytical SQL; lakehouses combine object storage flexibility with transactional table layers and analytical capabilities.

- Choose based on concurrency, governance, cost, and workload.
- Avoid copying data into many systems without a clear reason.

5. File Formats and Data Layout

File format affects scan performance, schema handling, compression, interoperability, and cost. Data engineers should understand CSV, JSON, Avro, Parquet, and table formats.

Key idea: For analytical workloads, columnar formats such as Parquet usually provide major advantages over row-oriented text formats.

CSV and JSON

CSV is simple and widely interoperable but weakly typed and ambiguous around quoting, nulls, and schemas. JSON is flexible and useful for nested data but can be verbose.

- Treat CSV schemas as explicit contracts.
- Validate JSON structures.
- Avoid repeatedly parsing raw text in downstream jobs.

Parquet

Parquet is a columnar format designed for analytical workloads. It supports typed columns, compression, and selective reads.

- Column pruning reduces I/O.
- Compression reduces storage and transfer.
- File sizing and partitioning still matter.

Avro and table formats

Avro is useful for row-oriented serialization and schema-aware data exchange. Table formats/layers such as Delta add transactional and metadata capabilities around files.

- Separate serialization format from table-management concerns.
- Use a format appropriate to the interchange and processing system.

6. Data Modeling Fundamentals

Data modeling defines how entities, relationships, facts, dimensions, keys, and attributes are represented. A good model makes data understandable and supports efficient queries.

Key idea: Always state the grain of a fact table: what exactly does one row represent?

OLTP vs analytical modeling

Operational systems often favor normalized schemas and transaction integrity. Analytical systems often use dimensional or denormalized models to simplify queries.

- Normalize to reduce update anomalies in transactional systems.
- Denormalize deliberately for analytical access patterns.

Facts and dimensions

Fact tables store events or measurements at a defined grain. Dimension tables describe business entities and provide descriptive context.

- Define grain before adding measures.
- Choose keys carefully.
- Avoid mixing grains in the same fact table.

Slowly changing dimensions

SCD techniques manage changes in descriptive attributes over time. Type 1 overwrites; Type 2 preserves history with versions/effective dates.

- Use Type 2 when historical reporting requires prior values.
- Define current-row logic clearly.
- Test late-arriving dimensions.

7. SQL for Data Engineers

SQL remains one of the most important tools for data engineering. Data engineers use it for extraction, validation, transformation, modeling, troubleshooting, and performance analysis.

Key idea: Strong SQL is not only syntax. It is understanding relational semantics, joins, nulls, aggregation, window functions, indexes, and query plans.

Core operations

SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY, UNION, CASE, CTEs, and subqueries form the foundation.

- Filter before expensive joins when logically equivalent.
- Understand inner vs outer joins.
- Treat NULL as unknown rather than as an ordinary value.

Window functions

Window functions calculate values across related rows without collapsing the result set. Common functions include ROW_NUMBER, RANK, LAG, LEAD, SUM OVER, and moving aggregates.

- Define PARTITION BY and ORDER BY explicitly.
- Use windows for deduplication, sequencing, and change detection.

Query plans

Execution plans reveal scans, seeks, joins, sorts, aggregation, and data movement. Performance tuning should use evidence from plans and runtime metrics.

- Do not assume an index is used because it exists.
- Avoid functions on indexed columns when they prevent efficient access.
- Measure after changes.

8. Data Pipelines and Orchestration

Orchestration coordinates tasks, dependencies, schedules, retries, parameters, and monitoring. It turns individual transformations into a reliable workflow.

Key idea: A pipeline is an operational product: it needs ownership, observability, retry policy, failure handling, and a defined SLA.

DAGs

Many orchestrators represent workflows as directed acyclic graphs where tasks have explicit dependencies.

- Independent tasks can run in parallel.
- Cycles indicate invalid dependency design.
- Keep tasks small enough to retry but meaningful enough to observe.

Scheduling

Schedules can be time-based, event-driven, dependency-driven, or manually triggered.

- Use event triggers when source arrival is the true dependency.
- Avoid overly frequent schedules that create overlapping runs.

Retries and idempotency

Retries help with transient failures. They become dangerous when a task is not idempotent and repeated execution creates duplicates or inconsistent state.

- Use deterministic output paths/keys.
- Write atomically when possible.
- Separate transient failures from data-quality failures.

Parameters

Parameterized pipelines allow the same logic to process different dates, environments, sources, or partitions.

- Validate parameters.
- Do not build SQL by unsafe string concatenation.
- Log the effective parameters for reproducibility.

9. Data Quality and Data Contracts

Data quality is the degree to which data meets defined expectations. A data contract makes important expectations explicit between producers and consumers.

Key idea: A pipeline that runs successfully but produces wrong data is a failed pipeline.

Quality dimensions

Common dimensions include accuracy, completeness, consistency, validity, uniqueness, timeliness, and integrity.

- Choose checks tied to business meaning.
- Not every dataset needs every check.
- Monitor trends, not only binary pass/fail status.

Contracts

A contract can specify schema, semantics, allowed values, ownership, freshness, and compatibility expectations.

- Breaking changes should be versioned and communicated.
- Schema evolution should be controlled.
- Consumers should not depend on undocumented fields.

Handling bad data

Bad records can be quarantined, rejected, corrected, or accepted with warnings depending on the use case.

- Preserve enough metadata to investigate.
- Avoid silently dropping records.
- Define thresholds for pipeline failure.

10. Incremental Processing, CDC, and Idempotency

Incremental processing reduces cost by processing only new or changed data. It depends on a reliable change signal such as a timestamp, sequence number, version, or CDC log.

Key idea: The central question is: if the pipeline runs twice for the same input, will the final state remain correct?

Watermarks

A watermark records progress through a source, such as the greatest processed timestamp or sequence value.

- Handle late-arriving records.
- Do not assume timestamps are unique.
- Store progress in durable state.

CDC

CDC streams changes from operational databases. Pipelines must interpret inserts, updates, and deletes correctly.

- Preserve change order where required.
- Use primary keys to identify target records.
- Handle schema changes separately from row changes.

Idempotent writes

An idempotent operation produces the same final result when applied repeatedly to the same logical input.

- MERGE/upsert patterns can help.
- Deterministic partition replacement can help.
- Avoid blindly appending data when reruns are possible.

11. Distributed Systems Fundamentals

Large-scale data engineering relies on distributed systems concepts: partitioning, replication, consistency, fault tolerance, network costs, serialization, and coordination.

Key idea: Distributed systems trade local simplicity for scale and failure tolerance. Network and coordination are real costs.

Partitioning

Partitioning divides data or work across nodes. Good partitioning balances load and minimizes communication.

- Hash partitioning distributes keys.
- Range partitioning supports ordered/range access.
- Hot keys can create skew.

Replication

Replication keeps multiple copies or replicas so systems can survive failures and serve reads.

- More replicas can improve availability but increase cost and write complexity.
- Understand the consistency model before assuming replicas are interchangeable.

Failure

Nodes, networks, disks, processes, and dependencies can fail. Production systems must recover or degrade predictably.

- Use retries with backoff for transient errors.
- Use timeouts to prevent indefinite waits.
- Make recovery observable.

12. Cloud Data Engineering on Azure

Azure provides services for storage, integration, compute, databases, analytics, identity, networking, and monitoring. A data engineer composes them into an architecture rather than using every service by default.

Key idea: A good Azure architecture is driven by requirements: latency, volume, security, cost, operational ownership, and skills.

Common building blocks

Azure Data Lake Storage provides object storage; Azure Data Factory provides orchestration/integration; Azure Databricks provides Spark/lakehouse compute; Synapse provides integrated analytics capabilities; Azure SQL Database provides managed relational storage.

- Use storage for durable data.
- Use compute for transformation.
- Use orchestration to coordinate.
- Use governance/identity across the platform.

Identity and security

Microsoft Entra ID, managed identities, Key Vault, private networking, RBAC, and service-specific controls support secure architectures.

- Prefer identity-based authentication.
- Separate environments.
- Minimize public endpoints where requirements demand private access.

13. Azure Data Lake Storage and Lakehouse Storage

Azure Data Lake Storage Gen2 provides scalable cloud storage for data-lake workloads. It integrates hierarchical namespace capabilities with Azure Blob Storage foundations.

Key idea: A data lake is a storage foundation; a lakehouse adds reliable table semantics, governance, and analytics capabilities.

Storage organization

Paths, containers, access policies, naming conventions, and lifecycle rules affect usability and cost.

- Organize around domains and data products where appropriate.
- Keep raw and curated areas distinct.
- Avoid embedding secrets in file names or paths.

Security

Azure RBAC and storage access controls can restrict access. Network controls can further limit connectivity.

- Use least privilege.
- Separate read/write identities.
- Audit access to sensitive datasets.

Lifecycle

Data can move to cooler/archival tiers or be deleted according to retention policies.

- Retention should be driven by business, legal, and recovery requirements.
- Do not delete data merely because it is old.

14. Warehouses, Lakehouses, and Data Marts

Warehouses and lakehouses both support analytics but differ in architecture, storage assumptions, and governance patterns. Data marts focus on a particular business domain or analytical workload.

Key idea: A platform can contain multiple serving layers, but every copy creates cost and consistency obligations.

Warehouse

A warehouse is optimized for structured analytical queries, concurrency, governance, and SQL workloads.

- Strong fit for BI and governed reporting.
- Schema and workload management are central.

Lakehouse

A lakehouse combines object storage with table formats and analytical compute, supporting multiple data types and workloads.

- Useful when one storage foundation serves engineering, analytics, and ML.
- Governance and table design remain critical.

Marts

A data mart is a curated subset organized for a business domain such as finance, sales, or operations.

- Marts can simplify BI consumption.
- Define ownership and refresh expectations.

15. Security, Governance, and Privacy

Data engineering systems often handle sensitive business and personal information. Security and governance must be embedded in architecture, pipelines, and operational processes.

Key idea: Security is a system property: identity, network, storage, application code, logging, and governance all contribute.

Least privilege

Grant only the permissions required for a task. Use groups/roles for maintainability.

- Separate developer and production access.
- Avoid shared administrator accounts.
- Review unused permissions.

Encryption

Encryption protects data at rest and in transit. Key management may use platform-managed or customer-managed keys depending on requirements.

- Protect keys as carefully as data.
- Use TLS for network communication.
- Document key ownership and rotation.

Privacy

Sensitive fields may require masking, tokenization, minimization, retention limits, and restricted access.

- Collect only what is needed.
- Avoid copying sensitive fields into lower-security environments.
- Make deletion/retention workflows testable.

Auditability

Audit logs and lineage help answer who accessed data, what changed, and how a dataset was produced.

- Keep operational logs long enough for investigations.
- Separate business data from audit metadata.

16. Observability and Monitoring

Observability lets operators understand system health and data behavior. For data platforms, this includes pipeline metrics, logs, traces where available, data quality, freshness, and cost.

Key idea: A green pipeline status is not enough. Monitor the data product itself.

Pipeline metrics

Track duration, success/failure, input/output row counts, bytes processed, retry count, and SLA compliance.

- Unexpected row-count drops can signal upstream issues.
- A successful run with zero output can still be a failure.

Data freshness

Freshness measures how recently a dataset was updated relative to its expected schedule.

- Set freshness expectations per dataset.
- Alert on meaningful breaches, not harmless delays.

Logs

Logs should identify the job, run, source, target, parameters, and key processing milestones without exposing secrets.

- Use structured logs where possible.
- Include correlation/run IDs.
- Avoid printing entire sensitive records.

Cost monitoring

Track compute hours, storage growth, data transfer, and expensive queries/jobs.

- Cost anomalies can reveal runaway workloads.
- Tag resources or workloads for ownership.

17. Testing Data Pipelines

Data pipelines need tests for code behavior, data transformations, schemas, quality rules, and integration points.

Key idea: Testing data is harder than testing a pure function because the expected result often depends on business rules and data distributions.

Unit tests

Test deterministic transformation functions with small fixtures.

- Keep tests fast.
- Test edge cases: nulls, duplicates, empty input, invalid values.

Data tests

Data tests validate properties such as uniqueness, ranges, non-null constraints, referential integrity, and freshness.

- Test business rules explicitly.
- Use representative samples.
- Fail loudly when critical assumptions break.

Integration tests

Integration tests validate connectivity, schema compatibility, permissions, and end-to-end behavior across services.

- Use isolated test resources.
- Avoid destructive tests against production.

Regression tests

A fixed set of representative datasets can detect unexpected changes after refactoring or dependency upgrades.

- Version test fixtures.
- Include expected row counts and key aggregates where stable.

18. CI/CD and Production Deployment

Continuous integration and deployment apply software-engineering discipline to data pipelines. Code, configuration, tests, and infrastructure should be reviewable and reproducible.

Key idea: The goal is not to automate every button click; it is to make deployments predictable and reversible.

Version control

Store source code, pipeline definitions, SQL, configuration templates, and documentation in version control.

- Use meaningful commits.
- Review changes.
- Tag production releases when appropriate.

Environments

Development, test/staging, and production environments isolate experimentation from live workloads.

- Parameterize environment-specific resources.
- Do not hard-code production paths into development code.
- Use separate identities where practical.

Deployment

Automated deployment can validate, test, package, and release data assets.

- Run tests before deployment.
- Use approvals for sensitive production changes.
- Keep rollback or forward-fix procedures documented.

19. Cost, Performance, and Reliability Trade-offs

Data-engineering architecture is a multi-objective optimization problem. Faster, cheaper, more reliable, and more flexible are not always the same design.

Key idea: State the requirement before choosing the technology. A daily batch job does not need a real-time architecture merely because streaming exists.

Latency vs cost

Lower latency often requires more continuously running compute, which can cost more than scheduled batch processing.

- Define the actual business latency requirement.
- Use event-driven designs when the value justifies them.

Reliability vs complexity

More components can provide specialized capabilities but also create more operational failure points.

- Prefer simple architectures when requirements allow.
- Make every component earn its place.

Performance

Optimize bottlenecks rather than all components. Storage layout, partitioning, query shape, network transfer, and compute sizing can all matter.

- Benchmark with representative data.
- Track cost per successful workload, not only runtime.

Reliability

Retries, idempotency, checkpoints, validation, and disaster recovery improve reliability but require design and testing.

- Run failure drills.
- Document recovery time and recovery point expectations.

20. End-to-End Data Engineering Reference Architecture

This final page connects the concepts into a practical architecture suitable for a portfolio project and for understanding how Azure data services fit together.

Key idea: A strong data-engineering project demonstrates the complete lifecycle: source → ingestion → storage → transformation → quality → serving → monitoring → governance.

Reference flow

Operational DB/API/files/events feed an ingestion layer. Raw data lands in durable storage. Transformations create standardized and business-ready datasets. Consumers access curated tables through SQL/BI/ML applications.

- Source systems → ingestion/CDC → raw storage.
- Raw → validation/standardization → curated.
- Curated → marts/serving → BI/ML/applications.
- Orchestration and monitoring surround the pipeline.

Design checklist

Before implementation, write down volume, latency, retention, schema, quality, security, recovery, cost, and ownership requirements.

- Define the grain of important datasets.
- Define idempotency and backfill behavior.
- Define SLAs/SLOs and data-quality rules.
- Choose technologies after requirements are clear.

Portfolio mapping

For your RAG learning project, these documents can also serve as a realistic knowledge base: Spark for distributed compute, Azure Databricks for lakehouse engineering, ADF for orchestration, Synapse for analytics, Azure SQL for relational data, and this document for the broader discipline.

- Later, we can ask the RAG system comparative questions.
- We can test source/page retrieval.
- We can evaluate whether answers cite the correct document and chunk.

Selected official sources

- <https://learn.microsoft.com/en-us/training/career-paths/data-engineer>
- <https://learn.microsoft.com/en-in/training/paths/get-started-data-engineering/>
- <https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-introduction>
- <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/data-lake>